NKIGym IR Guide: From Python to NKI Kernels

NKI Autotune

2026-03-03

NKIGym IR Guide

Running example: `tanh(a.T @ b)` where `a: [256, 128]`, `b: [256, 128]` –> result `[128, 128]`.

Layer	Representation	Code
User Function	Python source using `nkigym.*` ops	
	codegen	`function_to_program/`
GymProgram	`GymProgram` NamedTuple (immutable, hashable)	`ir/`, `transforms/`
	codegen	`program_to_nki/`
NKI	`nki.isa.*` / `nki.language.*` source	compiled by `neuronxcc`

Codegen = level changes (`function_to_program/`, `program_to_nki/`). **Transforms** = GymProgram –> GymProgram (same level, `transforms/`).

---

1. User Function (codegen: function –> GymProgram)

`source_to_program(source, kwargs) --> GymProgram` (in `function_to_program/`). `kwargs` maps parameter names to values — numpy arrays for tensor inputs, other Python objects for non-tensor arguments. Shapes are inferred from the arrays; `output_dtype` is inferred from input tensor dtypes (raises if they differ).

```
source_to_program(matmul_tanh, kwargs={"a": np.zeros((256, 128), dtype=np.float32),
                                       "b": np.zeros((256, 128), dtype=np.float32)})
```

1.1 User function

```
def matmul_tanh(a, b):
    c = nkigym.nc_matmul(a, b)
    return nkigym.activation(c, op=np.tanh)
```

1.2 Tiled function

K=256 exceeds tile limit 128, so codegen splits into 2 reduction tiles with accumulation:

```
def matmul_tanh(a, b):
    tile_0 = nkigym.nc_matmul(a[0:128, 0:128], b[0:128, 0:128])
    tile_1 = nkigym.nc_matmul(a[128:256, 0:128], b[128:256, 0:128],
                              acc=tile_0[0:128, 0:128])
    return nkigym.activation(tile_1[0:128, 0:128], op=np.tanh)
```

1.3 Tiled function with IO

Codegen wraps compute with explicit IO ops — `np.empty` (allocate), `nkigym.load` (load tile from HBM), `nkigym.store` (write tile to HBM). These are distinct from genuine numpy slicing. Indexing into on-chip results (SBUF/PSUM) uses plain slicing — the parser distinguishes this from HBM loads and emits `IndexingOp` in the IR.

```
def matmul_tanh(a, b):
    output = np.empty((128, 128), dtype=np.float32)

    tensor_0 = nkigym.load(a[0:128, 0:128])
    tensor_1 = nkigym.load(b[0:128, 0:128])
    tensor_2 = nkigym.nc_matmul(tensor_0[0:128, 0:128], tensor_1[0:128, 0:128])

    tensor_3 = nkigym.load(a[128:256, 0:128])
    tensor_4 = nkigym.load(b[128:256, 0:128])
    tensor_5 = nkigym.nc_matmul(tensor_3[0:128, 0:128], tensor_4[0:128, 0:128],
                                acc=tensor_2[0:128, 0:128])

    tensor_6 = tensor_5[0:128, 0:128]          # index into PSUM (not HBM load)
    tensor_7 = nkigym.activation(tensor_6[0:128, 0:128], op=np.tanh)

    nkigym.store(tensor_7[0:128, 0:128], output[0:128, 0:128])
    return output
```

Parsing this function mechanically produces the GymProgram with `AllocateOp`, `LoadOp`, `MatmulOp`, `IndexingOp`, `ActivationOp`, `StoreOp` statements.

Statement types: `AllocateOp` (allocate), `LoadOp` (load tile from HBM), `IndexingOp` (pure indexing into SBUF/PSUM), compute ops (`MatmulOp`, etc.), `StoreOp` (write tile to HBM).

---

2. GymProgram (in `ir/`)

Translation from function (section 1.3) to GymProgram is purely mechanical — each function statement maps 1:1 to a GymStatement, with `np.empty` becoming `AllocateOp`, `nkigym.load` becoming `LoadOp`, `nkigym.store` becoming `StoreOp`, and `nkigym.*` compute calls becoming their corresponding op class.

Both representations are directly callable: `matmul(a, b)` runs the function with numpy, `program(a=a, b=b)` interprets the IR by dispatching `stmt.op.simulate()` op by op. Same numpy result either way — useful for verification.

IR types:

```
class TensorRef(NamedTuple):
    name: str                           # variable name
    shape: tuple[int, ...]              # tensor shape
    slices: tuple[tuple[int, int], ...] # per-axis (start, stop), always present

class GymStatement(NamedTuple):
    op: type[GymOp]                     # the GymOp subclass itself (e.g. MatmulOp)
    kwargs: tuple[tuple[str, Any], ...] # (name, value) pairs --- Python objects, not strings
    output: TensorRef

class GymProgram(NamedTuple):
    name: str
    kwargs: dict[str, Any]              # input arrays + non-tensor args (the original caller kwargs)
```

```
    stmts: tuple[GymStatement, ...]
    return_var: str
    output_dtype: type                    # inferred from input tensor dtypes (must all match)
```

Running example parsed from section 1.3:

```
GymProgram(
    name="matmul_tanh",
    kwargs={"a": np.zeros((256, 128), dtype=np.float32),
            "b": np.zeros((256, 128), dtype=np.float32)},
    stmts=(
        GymStatement(AllocateOp, (("dtype", np.float32),),
            TensorRef("output", (128, 128), ((0, 128), (0, 128)))),

        GymStatement(LoadOp,
            (("src", TensorRef("a", (256, 128), ((0, 128), (0, 128)))),),
            TensorRef("tensor_0", (128, 128), ((0, 128), (0, 128)))),
        GymStatement(LoadOp,
            (("src", TensorRef("b", (256, 128), ((0, 128), (0, 128)))),),
            TensorRef("tensor_1", (128, 128), ((0, 128), (0, 128)))),
        GymStatement(MatmulOp,
            (("stationary", TensorRef("tensor_0", (128, 128), ((0, 128), (0, 128)))),
             ("moving",     TensorRef("tensor_1", (128, 128), ((0, 128), (0, 128))))),
            TensorRef("tensor_2", (128, 128), ((0, 128), (0, 128)))),

        GymStatement(LoadOp,
            (("src", TensorRef("a", (256, 128), ((128, 256), (0, 128)))),),
            TensorRef("tensor_3", (128, 128), ((0, 128), (0, 128)))),
        GymStatement(LoadOp,
            (("src", TensorRef("b", (256, 128), ((128, 256), (0, 128)))),),
            TensorRef("tensor_4", (128, 128), ((0, 128), (0, 128)))),
        GymStatement(MatmulOp,
            (("stationary", TensorRef("tensor_3", (128, 128), ((0, 128), (0, 128)))),
             ("moving",     TensorRef("tensor_4", (128, 128), ((0, 128), (0, 128)))),
             ("acc",        TensorRef("tensor_2", (128, 128), ((0, 128), (0, 128))))),
            TensorRef("tensor_5", (128, 128), ((0, 128), (0, 128)))),

        GymStatement(IndexingOp,
            (("src", TensorRef("tensor_5", (128, 128), ((0, 128), (0, 128)))),),
            TensorRef("tensor_6", (128, 128), ((0, 128), (0, 128)))),
        GymStatement(ActivationOp,
            (("data", TensorRef("tensor_6", (128, 128), ((0, 128), (0, 128)))),
             ("op", np.tanh)),
            TensorRef("tensor_7", (128, 128), ((0, 128), (0, 128)))),

        GymStatement(StoreOp,
            (("src", TensorRef("tensor_7", (128, 128), ((0, 128), (0, 128)))),
             ("dst", TensorRef("output",   (128, 128), ((0, 128), (0, 128))))),
            TensorRef("output", (128, 128), ((0, 128), (0, 128)))),
    ),
    return_var="output",
    output_dtype=np.float32,
)
```

`op` stores the `GymOp` subclass directly — dispatch is `stmt.op.simulate(...)` / `stmt.op.to_nki(stmt,`

`ctx)` with no registry lookup. `kwargs` replaces the old `params` + `input_shapes` fields — param names are `kwargs.keys()`, shapes/dtype are read from the numpy arrays. Graph deduplication keys on `stmts` only — search optimizes a single specialized kernel at a time, so `kwargs` is constant across all variants.

---

3. Transforms (GymProgram –> GymProgram, in `transforms/`)

```
class Transform(ABC):
    def analyze_ir(self, program: GymProgram) -> list[Any]: ...
    def transform_ir(self, program: GymProgram, option: Any) -> GymProgram: ...
```

OperandMergeTransform — finds pairs of the same operator and tries to merge their operands into a wider operation. No distinction between load/compute/store — the same logic applies uniformly. Checked against `stmt.op.tile_limits`.

```
Before:  tensor_1 = load(b[0:128, 0:128])
         tensor_4 = load(b[0:128, 128:256])
After:   tensor_1 = load(b[0:128, 0:256])       # tensor_4 eliminated, consumers remapped

Before:  tensor_2 = nc_matmul(stationary=t0, moving=t1[0:128, 0:128])
         tensor_5 = nc_matmul(stationary=t3, moving=t1[0:128, 128:256])
After:   tensor_2 = nc_matmul(stationary=t0, moving=t1[0:128, 0:256])

Before:  store(tensor_2[0:128, 0:128], output[0:128, 0:128])
         store(tensor_5[0:128, 0:128], output[0:128, 128:256])
After:   store(tensor_2[0:128, 0:256], output[0:128, 0:256])
```

DataReuseTransform — eliminates redundant loads:

```
Before:  tensor_0 = a[0:128, 0:128]    # subgraph 1
         tensor_3 = a[0:128, 0:128]    # subgraph 2 (redundant)
After:   tensor_0 = a[0:128, 0:128]    # tensor_3 eliminated, uses rewritten
```

Search

`search(func, transforms, ...) --> SearchResults`

Phase	Operation	I/O
Codegen	`function_to_program/`	User Function –> GymProgram
Transforms	Find opportunities	GymProgram –> list[option]
	Apply transform	GymProgram –> GymProgram
	Deduplicate	`stmts` hashing
	Verify	`program(**kwargs)` via `stmt.op.simulate()`
Codegen	`program_to_nki/`	GymProgram –> NKI source
Compile	neuronxcc	NKI source –> NEFF
Benchmark	run_on_hardware	NEFF –> metrics

Search explores thousands of GymProgram variants (cheap `__call__` evaluation). Only qualifying variants cross to NKI and hardware.

---

4. NKI (codegen: GymProgram –> NKI)

`lower_to_nki(GymProgram) --> str` (in `program_to_nki/`). Dispatches `stmt.op.to_nki(stmt, ctx)` directly, then `roll_loops()` + `_use_affine_range()` compress repeating blocks into loops.

Lowering rules:

GymStatement op	NKI output
`AllocateOp`	`nl.ndarray(..., buffer=nl.shared_hbm)`
`LoadOp`	alloc SBUF + `nisa.dma_copy` (HBM only)
`IndexingOp`	pure indexing (SBUF/PSUM)
`MatmulOp` (first)	alloc PSUM + `nisa.nc_matmul`
`MatmulOp` (acc=)	reuse PSUM via alias
`ActivationOp`	`nisa.activation(...)` on SBUF
`StoreOp` from SBUF	`nisa.dma_copy` to HBM
`StoreOp` from PSUM	`nisa.tensor_copy` –> SBUF staging –> `nisa.dma_copy` to HBM

Memory: HBM (input/output) –> SBUF (tiles) –> PSUM (accumulation, float32).

Running example `tanh(a.T @ b)`, single output tile with 2 reduction tiles:

```python
@nki.jit
def matmul_tanh(a, b):
    output = nl.ndarray((128, 128), dtype=nl.float32, buffer=nl.shared_hbm)

    # Reduction tile 0: K=[0:128]
    tensor_0 = nl.ndarray((128, 128), dtype=nl.float32, buffer=nl.sbuf)
    nisa.dma_copy(dst=tensor_0[0:128, 0:128], src=a[0:128, 0:128])
    tensor_1 = nl.ndarray((128, 128), dtype=nl.float32, buffer=nl.sbuf)
    nisa.dma_copy(dst=tensor_1[0:128, 0:128], src=b[0:128, 0:128])
    tensor_2 = nl.ndarray((128, 128), dtype=nl.float32, buffer=nl.psum)
    nisa.nc_matmul(dst=tensor_2[0:128, 0:128],
                   stationary=tensor_0[0:128, 0:128],
                   moving=tensor_1[0:128, 0:128])

    # Reduction tile 1: K=[128:256], accumulate into tensor_2
    tensor_3 = nl.ndarray((128, 128), dtype=nl.float32, buffer=nl.sbuf)
    nisa.dma_copy(dst=tensor_3[0:128, 0:128], src=a[128:256, 0:128])
    tensor_4 = nl.ndarray((128, 128), dtype=nl.float32, buffer=nl.sbuf)
    nisa.dma_copy(dst=tensor_4[0:128, 0:128], src=b[128:256, 0:128])
    nisa.nc_matmul(dst=tensor_2[0:128, 0:128],
                   stationary=tensor_3[0:128, 0:128],
                   moving=tensor_4[0:128, 0:128])

    # IndexingOp (PSUM) + ActivationOp (needs SBUF, so PSUM-->SBUF staging)
    tensor_6 = nl.ndarray((128, 128), dtype=nl.float32, buffer=nl.sbuf)
    nisa.tensor_copy(dst=tensor_6[0:128, 0:128], src=tensor_2[0:128, 0:128])
    tensor_7 = nl.ndarray((128, 128), dtype=nl.float32, buffer=nl.sbuf)
    nisa.activation(dst=tensor_7[0:128, 0:128],
                    data=tensor_6[0:128, 0:128], op=nl.tanh)

    # StoreOp
    nisa.dma_copy(dst=output[0:128, 0:128], src=tensor_7[0:128, 0:128])
    return output
```

With larger output tilings (e.g., 256x256 with 2x2 parallel tiles), `roll_loops()` compresses repeating tile blocks into `nl.affine_range` loops.